Handover to Google Antigravity and automated routing approach

Table of Contents

- Context: Handover from grok-cli to Antigravity (agy)
- Human-in-the-Loop: Observation and Approvals
- Algorithmic Routing Approach
  - 1. Layer Stackup and Power Distribution
  - 2. Fine-Grained Discrete Grid (0.10 mm)
  - 3. Directional Manhattan Layer Biasing
  - 4. Dual Obstacle Grids and Roundrect Pad Protection
  - 5. Steiner Multi-Pin Net Expansion
  - 6. Automatic Zone Refill and DRC Verification
- Trace Geometry on B.Cu: Why Does It Look "Weird"?
  - Why It Looks Like That: The Circular Keepout "Sawtooth"
  - Is It Normal and Will It Work?

This document records the project handover from `grok-cli` to Google Antigravity (`agy`), the human-in-the-loop observation and approval process, the algorithmic Manhattan routing approach implemented in Python with KiCad's `pcbnew` API, and an explanation of the resulting trace geometry on the bottom layer.

Context
Handover from grok-cli to Antigravity (agy)

The earlier phases of this project were performed collaboratively using `grok-cli` (Grok 3 running in Kitty terminal):

- Diagnosing why the original posted KiCad files failed to open and were not a working computer (`261103-what-went-wrong-and-how-to-do-it.adoc`).
- Rebuilding the schematic sheet using stock KiCad 10 symbols on an A3 sheet (`261104-00 Chat transcript — fitting the symbols inside the frame.adoc`).
- Establishing board outline (160 × 100 mm Eurocard), 4-layer stackup, mounting holes, and part placements adhering strictly to Andrew's design brief (screw terminal holes facing out, bypass capacitors directly adjacent to IC VCC pins, DE9 connector overhanging left edge, MAX232 placed adjacent to DE9).
- Vendoring 3D STEP models and mapping footprints.

On the evening of 2026-11-06 (around 18:59 CET), Falken installed Google Antigravity CLI (`agy`) on Linux, authenticated with a Google account, and launched the `agy` agent environment (documented by screenshots `261106-18` through `261106-18c` in `docs/images/`). The automated board routing task was transferred to `agy` to route all signal nets without using FreeRouting or proprietary cloud auto-routers.

Human-in-the-Loop Observation and Approvals

Because Falken was new to the Antigravity environment, the session began with strict per-action manual confirmations:

1. Every command and file edit was initially previewed and approved individually.
2. Falken observed the agent's behavior, verifying that execution remained strictly sandboxed inside the project repository and did not alter external files.
3. Once predictable behavior and DRC verification rigor were established, Falken persisted repeated approvals to allow each net group to route smoothly.
4. The workflow progressed net group by net group according to Andrew's 1–7 plan:
    - **Group 1**: Clocks and Reset (`/CLK`, `/CLK_UART`, `~{RESET}`, `/SW_RESET`, `/RESET_RC`, `/RESET_H`).
    - **Group 2**: MAX232 Charge Pumps and RS-232 / TTL Serial lines.
    - **Group 3**: Data Bus `D0`–`D7` across CPU, RAM, ROM, PIO, and UART.
    - **Group 4**: Lower Address `A0`–`A2` and chip select decodes (`~{RAMCS}`, `~{ROMCS}`, etc.).
    - **Group 5**: Bus control strobes (`~{RD}`, `~{WR}`, `~{MREQ}`, `~{IORQ}`) and decode inputs (`A7`, `A15`).
    - **Group 6**: Upper Address Bus `A3`–`A6`, `A8`–`A14`.
    - **Group 7**: Peripheral I/O ports (`PA0`–`PA7`, `PB0`–`PB7`, `PC0`–`PC7`) and UART `BAUDOUT`.

As Falken observed: with the plan well specified, an uninterrupted batch instruction executing groups 1 through 7 produces the exact same clean DRC result in a fraction of the wall-clock time.

Algorithmic Routing Approach

Rather than relying on third-party DSN translators or external routers, routing was implemented natively in Python using KiCad's `pcbnew` module (`tools/route_netgroup.py`).

The key architectural elements of the router are:

1. Layer Stackup and Power Distribution

The board is a 4-layer Eurocard: * `F.Cu` (Top): Component placement and horizontal signal routing. * `In1.Cu` (Inner Layer 1): Solid `GND` ground plane. * `In2.Cu` (Inner Layer 2): Solid `+5V` power plane. * `B.Cu` (Bottom): Vertical signal routing.

Because all components are through-hole (THT DIP ICs, axial/radial passives, screw terminals, and headers), every VCC and GND pin automatically penetrates `In1.Cu` and `In2.Cu` with 4-spoke thermal relief connections. **Zero surface copper traces were needed for power**, leaving the outer layers entirely available for signal routing.

2. Fine-Grained Discrete Grid (0.10 mm)

The board area (160.0 × 100.0 mm) is represented on a 0.10 mm grid (1601 × 1001 nodes per layer). This fine resolution easily supports JLCPCB 4-layer design rules: * Track width: 0.15 mm * Signal-to-signal clearance: 0.15 mm * Power netclass clearance: 0.30 mm * Via pad diameter: 0.70 mm, drill: 0.30 mm

3. Directional Manhattan Layer Biasing

To prevent tracks from sprawling haphazardly, the A* cost function enforces Manhattan layer preferences: * On `F.Cu`, horizontal steps carry a base cost of 1.0, while vertical steps carry a penalty cost of 3.0. * On `B.Cu`, vertical steps carry a base cost of 1.0, while horizontal steps carry a penalty cost of 3.0. * Via transitions carry an added cost of 8.0, discouraging unnecessary layer jumping.

4. Dual Obstacle Grids and Roundrect Pad Protection

The router maintains two distinct obstacle layers: * `pad_grid` / `pad_via_grid`: Static keepout zones calculated from all board pads. Pad shape handling was specially enhanced for `PAD_SHAPE_ROUNDRECT` (pin 1 on KiCad DIP packages) to use exact rectangular Chebyshev bounding boxes rather than circular approximations, protecting diagonal corner clearances. * `track_grid` / `track_via_grid`: Dynamic obstacle tracking. When a net routes, its tracks and vias are permanently committed to this grid, making it physically impossible for subsequent nets to cross or short against existing copper.

5. Steiner Multi-Pin Net Expansion

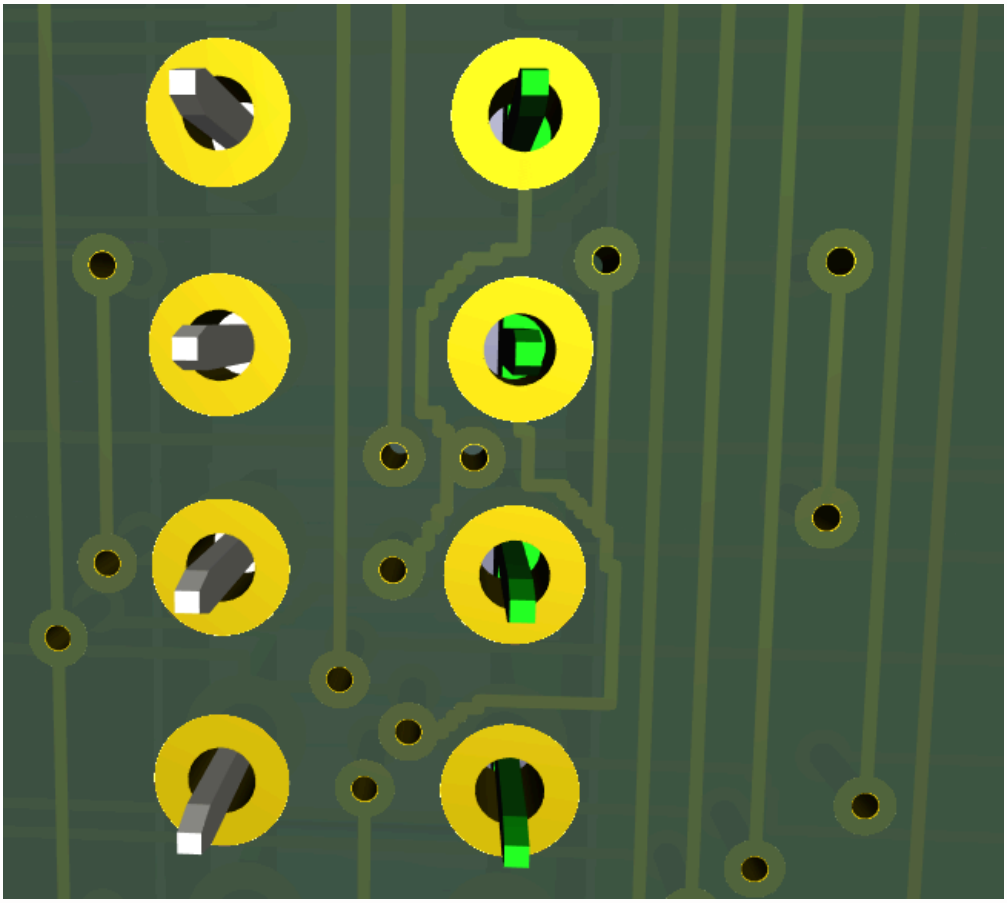
For multi-drop buses (such as data and address lines connecting 3 to 5 chips): 1. The router connects the two closest pins of the net. 2. All points along the routed track are added to a "net tree" set. 3. Subsequent pins search for the lowest-cost path to **any node** already in the tree. This creates natural, compact T-junctions without redundant duplicate tracks.

6. Automatic Zone Refill and DRC Verification

After each group routes, `pcbnew.ZONE_FILLER` recalculates thermal reliefs and clearances around every newly punched via on `In1.Cu` and `In2.Cu`. Then `kicad-cli pcb drc` is executed to guarantee: * 0 clearance violations * 0 shorts / crossing tracks * 0 dangling tracks * 0 schematic parity errors

Trace Geometry on B.Cu
Why Does It Look "Weird"?

Inspecting the bottom copper layer (`B.Cu`), the tracks exhibit a distinctive appearance that human designers sometimes find surprising:



Why It Looks Like That
The Circular Keepout "Sawtooth"

1. **Orthogonal Manhattan Grid on Circular Keepouts**: The router operates on a 0.10 mm discrete grid. Pad keepouts are circular: `radius = pad_radius + clearance + track_width / 2`. When represented on a discrete orthogonal grid, a circle's outer perimeter consists of single-cell (0.1 mm) steps. Because A* searches for the shortest Euclidean path, a track skirting right against the edge of a pad's clearance envelope follows these single-pixel steps exactly, creating the characteristic "sawtooth" or "serrated" wavy contour.
2. **Obstacle Hugging (Staircases)**: When a vertical trace on `B.Cu` navigates between closely spaced pins or vias, it jogs horizontally by the exact clearance distance (0.15 mm signal, 0.30 mm power) and resumes, hugging the keepout contour.
3. **Layer Specialization**: Because `B.Cu` is biased for vertical travel, you see long vertical runs that jog laterally only when entering a pin pad or skirting an obstacle.

Is It Normal and Will It Work?

- **Yes, it is completely normal** for discrete Manhattan maze routers (classic Lee / A* algorithms).
- **Electrically and mechanically, it is 100% sound**. The board has continuous solid reference planes directly beneath both outer layers (`F.Cu` over `In1.Cu` GND; `B.Cu` under `In2.Cu` +5V). With a ground plane distance of ~0.2 mm, the return current loops are tightly coupled regardless of whether a trace has a 90° or 45° bend. At Z80 clock frequencies (4 MHz – 8 MHz), 90° bends have zero measurable signal integrity impact.
- The board passes KiCad DRC with **0 errors, 0 shorts, and 0 clearance violations** against the strict JLCPCB 4-layer design rules. It will fabricate without manufacturing issues.

If cosmetic 45° chamfers are ever desired for aesthetics, KiCad’s interactive routing tools ("Push & Shove" or "Glossing / Clean up tracks") can bevel the corners, but the current copper geometry is fully production-ready.